FD05-EPIS-Informe-ProyectoFinal

UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA

Escuela Profesional de Ingenieria de Sistemas

Informe Final

Proyecto *EPIS Analytics*

Curso: *Inteligencia de negocios*

Docente: *Mag. Patrick Cuadros Quiroga*

Integrantes:

Enzo Leonel Laqui Luyo (2022073907)

Steven Christopher Yizuka Baldeón (2002023628)

Tacna - Peru

2026

CONTROL DE VERSIONES					
Version	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	Equipo del proyecto	Equipo del proyecto	Docente del curso	25/04/2026	Version inicial del informe final del proyecto

Sistema *EPIS Analytics*

Informe Final del Proyecto

Version *1.0*

Indice General

1. Antecedentes
2. Planteamiento del Problema
3. Objetivos
4. Marco Teorico
5. Desarrollo de la Solucion
6. Cronograma
7. Presupuesto
8. Conclusiones
9. Recomendaciones
10. Bibliografia
11. Webgrafia
12. Anexos

1. Antecedentes

La Escuela Profesional de Ingenieria de Sistemas de la Universidad Privada de Tacna requiere herramientas que apoyen el analisis de informacion academica y digital de sus estudiantes. En el contexto del curso de Inteligencia de negocios, se planteo el desarrollo de un sistema web capaz de centralizar indicadores simulados de presencia digital en redes sociales y presentarlos mediante dashboards.

El proyecto inicio como un prototipo local por capas en Python, con modelos, repositorios, servicios, controladores y una vista Streamlit. Posteriormente, el alcance evoluciono hacia una arquitectura cliente-servidor mas moderna, separando el frontend Angular del backend FastAPI, manteniendo la persistencia en SQLite para el prototipo y agregando autenticacion basica con token.

Como parte del desarrollo documental se elaboraron los siguientes entregables:

Entregable	Proposito	Estado
FD01 - Informe de Factibilidad	Evalua viabilidad tecnica, economica, operativa, legal, social y ambiental; incluye Terraform e Infracost	Completado
FD02 - Documento de Vision	Define posicionamiento, usuarios, caracteristicas, Wiki y RoadMap	Completado
FD03 - Especificacion de Requerimientos	Documenta historias de usuario, criterios de aceptacion y escenarios de prueba	Completado
FD04 - Arquitectura de Software	Presenta diagramas obtenidos desde codigo, base de datos e infraestructura	Completado
FD05 - Informe Final	Consolida el proyecto, resultados, presupuesto, conclusiones y anexos	En este documento

El resultado final es **EPIS Analytics**, una aplicacion web para analizar publicaciones, alcance e interacciones simuladas de estudiantes EPIS en LinkedIn, Instagram y YouTube, considerando ciclos academicos del I al X.

2. Planteamiento del Problema

2.1. Problema

Actualmente, la informacion relacionada con la actividad digital de los estudiantes puede encontrarse dispersa entre varias redes sociales. LinkedIn, Instagram y YouTube representan espacios donde se evidencia participacion academica, profesional, divulgativa o comunicacional, pero no existe una herramienta centralizada que permita observar indicadores de manera ordenada.

Esta dispersion dificulta:

Dificultad	Efecto
Revision manual de redes sociales	Consume tiempo y reduce la comparabilidad
Ausencia de indicadores consolidados	Limita la toma de decisiones basada en datos
Falta de filtros por red social	Impide analizar desempeno especifico por plataforma
Informacion no segmentada por ciclo	Reduce la lectura academica del avance estudiantil
Acceso sin control	Riesgo de exponer informacion que deberia consultarse de forma protegida

En consecuencia, se plantea una solucion de inteligencia de negocios que permita visualizar informacion simulada de manera clara, filtrable y protegida mediante login.

2.2. Justificacion

El proyecto se justifica porque permite aplicar conceptos de inteligencia de negocios, arquitectura de software, desarrollo web, pruebas automatizadas, infraestructura como codigo y analisis de costos en un caso academico concreto.

Desde el punto de vista institucional, EPIS Analytics aporta una base para analizar datos relacionados con estudiantes de la Escuela Profesional de Ingenieria de Sistemas. Aunque la version actual usa datos simulados, el diseno deja abierta la posibilidad de integrar datos reales en una fase posterior, previa evaluacion legal, tecnica y de privacidad.

La solucion propuesta aporta valor porque:

Aporte	Descripcion
Centralizacion	Reune metricas de publicaciones, alcance e interacciones en una sola plataforma
Analisis visual	Usa graficos para facilitar interpretacion de datos
Segmentacion	Permite filtrar por estudiante, red social y vista global
Seguridad basica	Protege dashboards con login y token
Trazabilidad	Relaciona requerimientos, pruebas, arquitectura y costos
Escalabilidad futura	Puede migrar de SQLite a una base de datos productiva y de datos simulados a fuentes reales

2.3. Alcance

El alcance desarrollado comprende:

- Backend FastAPI con endpoints REST para salud, estudiantes, login y dashboards.
- Frontend Angular con vistas de login, dashboard personal y dashboard global.
- Persistencia local SQLite con tablas de usuarios y publicaciones.
- Generacion de datos simulados para estudiantes EPIS de ciclos I al X.
- Consideracion de LinkedIn, Instagram y YouTube como redes sociales iniciales.
- Filtros por red social con opcion general "Todas".

- Graficos mediante ECharts.
- Login basico con usuario, contrasena, hash y token.
- Rutas privadas protegidas mediante guard e interceptor en Angular.
- Pruebas unitarias backend y frontend.
- Modulo Terraform para infraestructura propuesta en AWS.
- Analisis de costos con Infracost.

Queda fuera del alcance actual:

- Consumo real de APIs de LinkedIn, Instagram o YouTube.
- Despliegue real en AWS mediante `terraform apply`.
- Integracion directa con Power BI.
- Gestion avanzada de roles, recuperacion de contrasena o administracion de usuarios.
- Uso productivo de datos personales reales sin consentimiento y evaluacion legal.

3. Objetivos

3.1. Objetivo general

Desarrollar un sistema web de inteligencia de negocios para visualizar y analizar indicadores simulados de actividad social de estudiantes EPIS, permitiendo consultar metricas personales y globales por red social, con una arquitectura cliente-servidor documentada, testeable y proyectada a infraestructura cloud.

3.2. Objetivos especificos

ID	Objetivo especifico	Resultado obtenido
OE-01	Definir la factibilidad tecnica, economica y operativa del proyecto	FD01 completado con analisis Terraform e Infracost
OE-02	Documentar la vision, alcance, usuarios y RoadMap del sistema	FD02 completado e incorporado a la Wiki del repositorio
OE-03	Especificar requerimientos mediante historias de usuario	FD03 completado con GitHub Issues HU-01 a HU-09
OE-04	Implementar backend por capas y API REST	FastAPI, servicios, repositorios y modelos implementados
OE-05	Implementar frontend web separado	Angular con login, dashboard personal y dashboard global
OE-06	Incorporar graficos y filtros por red social	ECharts y filtros para LinkedIn, Instagram, YouTube y Todas
OE-07	Proteger vistas privadas	Login con token, guard e interceptor
OE-08	Documentar arquitectura desde artefactos reales	FD04 completado con diagramas de codigo, base de datos e infraestructura
OE-09	Validar funcionalidad mediante pruebas	Pytest, pruebas Angular, build y validacion Terraform ejecutados

ID	Objetivo especifico	Resultado obtenido
		correctamente

4. Marco Teorico

4.1. Inteligencia de negocios

La inteligencia de negocios permite transformar datos en informacion util para apoyar la toma de decisiones. En el proyecto, los datos simulados de publicaciones, alcance e interacciones se transforman en indicadores visuales que permiten observar tendencias por estudiante, ciclo y red social.

4.2. Dashboard analitico

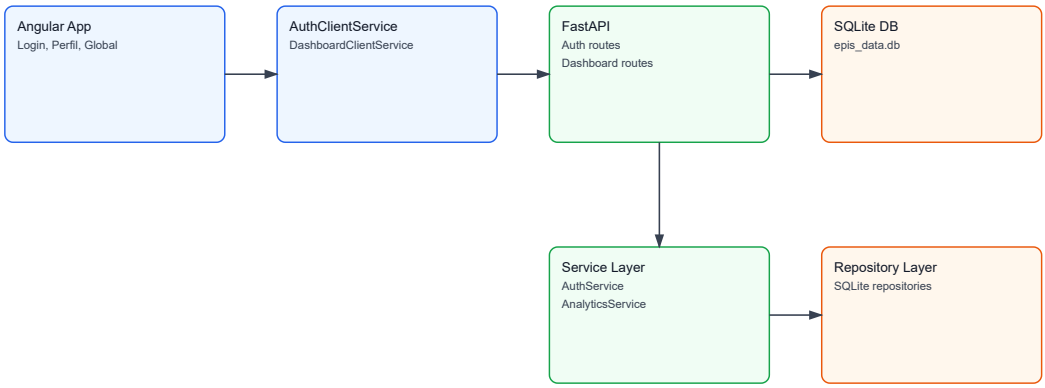
Un dashboard presenta informacion relevante mediante metricas, tablas y graficos. EPIS Analytics utiliza dos niveles:

Dashboard	Enfoque	Usuario principal
Perfil personal	Indicadores de un estudiante especifico	Estudiante EPIS
Impacto global EPIS	Indicadores agregados por ciclo, red social y ranking	Docente o responsable academico

4.3. Arquitectura cliente-servidor

La arquitectura cliente-servidor separa la interfaz de usuario del procesamiento de datos. En EPIS Analytics, Angular funciona como cliente web y FastAPI como servidor API. Esta separacion permite mejorar mantenibilidad, pruebas, despliegue y escalabilidad futura.

Diagrama de componentes



4.4. Patron Repository

El patron Repository encapsula el acceso a datos y evita que la logica de negocio dependa directamente del motor de base de datos. En el proyecto se usan repositorios SQLite para publicaciones y usuarios, permitiendo que en el futuro se pueda migrar a PostgreSQL u otro motor con menor impacto.

4.5. API REST

Una API REST expone recursos mediante endpoints HTTP. El backend FastAPI publica endpoints para autenticacion, consulta de estudiantes y dashboards. Esto permite que el frontend Angular consuma informacion de manera desacoplada.

4.6. Infraestructura como codigo

Terraform permite definir infraestructura cloud mediante archivos versionables. En el proyecto se declaran recursos AWS como EC2, Security Group, S3, CloudFront y politicas asociadas. Infracost complementa el analisis al estimar costos antes de desplegar.

5. Desarrollo de la Solucion

5.1. Analisis de Factibilidad

La factibilidad fue desarrollada en el documento FD01 y se resume en la siguiente matriz:

Dimension	Resultado	Sustento
Tecnica	Factible	Se usan Angular, FastAPI, SQLite, ECharts, Pytest, Terraform e Infracost
Economica	Factible	Herramientas sin licencia directa y costo cloud base estimado de USD 9.19 mensuales
Operativa	Factible	Flujo simple: login, consulta personal, consulta global y filtros
Social	Factible	Apoya analisis academico y cultura de decisiones basadas en datos
Legal	Factible para prototipo	Usa datos simulados; datos reales requieren consentimiento y revision normativa
Ambiental	Factible	Propuesta cloud minima y control de recursos mediante Terraform

5.1.1. Resultado de costos con Terraform e Infracost

La infraestructura propuesta considera un despliegue basico:

Componente	Recurso propuesto	Proposito
Backend	EC2 t3.micro	Ejecutar API FastAPI
Disco	EBS gp3 20 GB	Almacenamiento de la instancia
Frontend	S3	Hospedar archivos estaticos Angular
Distribucion	CloudFront	Entrega del frontend con HTTPS y cache
Seguridad	Security Group	Controlar acceso SSH, HTTP y API

El ultimo analisis local con Infracost arrojo:

Concepto	Estimacion
EC2 t3.micro	USD 7.59 mensual
EBS gp3 20 GB	USD 1.60 mensual
S3 y CloudFront	Variable segun uso
Total base mensual	**USD 9.192**

Con un tipo de cambio referencial de S/ 3.75, el costo base mensual aproximado es **S/ 34.47**.

5.2. Tecnologia de Desarrollo

Capa	Tecnologia	Uso
Frontend	Angular	Interfaz web, rutas, formularios y componentes
Graficos	ECharts	Visualizacion de metricas, rankings y series
Cliente HTTP	Angular services	Consumo de endpoints REST
Seguridad frontend	AuthGuard e Interceptor	Proteccion de rutas y envio de token
Backend	FastAPI	API REST
Dominio	Python	Modelos, servicios y reglas de negocio
Persistencia	SQLite	Base de datos local del prototipo
Repositorios	Repository Pattern	Abstraccion de acceso a datos
Pruebas backend	Pytest	Pruebas unitarias y de rutas API
Pruebas frontend	Angular Test Runner / Vitest	Validacion de componentes frontend
Infraestructura	Terraform	Declaracion de recursos AWS
Costos	Infracost	Estimacion de costos cloud
Documentacion	Markdown, PlantUML, GitHub Wiki	Informes, diagramas y soporte del proyecto

5.3. Metodologia de implementacion

El proyecto se desarrollo con un enfoque incremental. Cada entrega documental y tecnica permitio cerrar una parte del sistema y validar su avance:

Iteracion	Actividades	Evidencia
1	Modelo, repositorios y servicios base	Paquetes `models`, `repositories` y `services`
2	Controladores, vista Streamlit y pruebas	`controllers`, `views`, `app.py`, `tests`
3	Migracion a arquitectura web separada	`api` con FastAPI y `epis-web` con Angular
4	Login basico y rutas protegidas	`auth_routes.py`, `auth_service.py`, `auth.guard.ts`, `auth.interceptor.ts`
5	Graficos, filtros y ajustes visuales	Dashboards Angular con ECharts
6	Documentacion de factibilidad, vision, SRS y arquitectura	FD01, FD02, FD03 y FD04
7	Infraestructura y costos	`infra/terraform`, validacion Terraform e Infracost

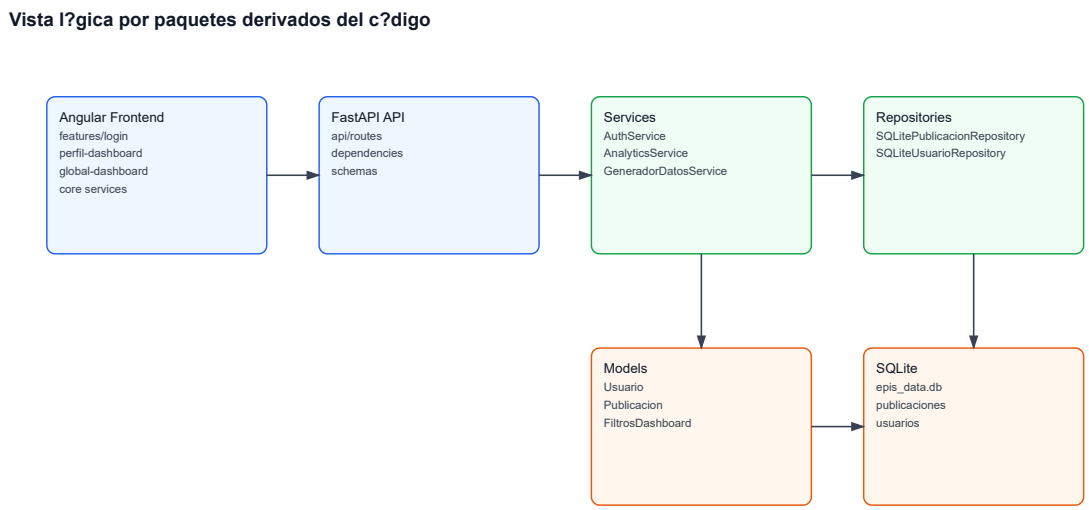
Iteracion	Actividades	Evidencia
8	Informe final	FD05

5.4. Requerimientos implementados

ID	Requerimiento	Estado
RF-01	Iniciar sesion con usuario, contrasena y token	Implementado
RF-02	Consultar dashboard personal	Implementado
RF-03	Filtrar dashboard personal por red social	Implementado
RF-04	Consultar dashboard global	Implementado
RF-05	Filtrar dashboard global por LinkedIn, Instagram o YouTube	Implementado
RF-06	Consultar lista de estudiantes	Implementado
RF-07	Proteger rutas privadas con token	Implementado
RF-08	Visualizar metricas mediante graficos	Implementado
RF-09	Validar infraestructura y costos	Implementado

5.5. Arquitectura implementada

La arquitectura final es cliente-servidor y mantiene separacion por responsabilidades:



Elemento	Ruta principal	Responsabilidad
Angular	`epis-web/src/app`	Interfaz, rutas, login, dashboards y graficos

Elemento	Ruta principal	Responsabilidad
FastAPI	`api`	Endpoints REST, CORS, autenticacion y orquestacion
Servicios	`services`	Reglas de negocio y calculo de indicadores
Repositorios	`repositories`	Acceso a SQLite
Modelos	`models`	Entidades y filtros del dominio
Base de datos	`epis_data.db`	Publicaciones y usuarios del prototipo
Infraestructura	`infra/terraform`	Recursos cloud propuestos

5.6. Base de datos

La base de datos local SQLite contiene dos tablas principales:

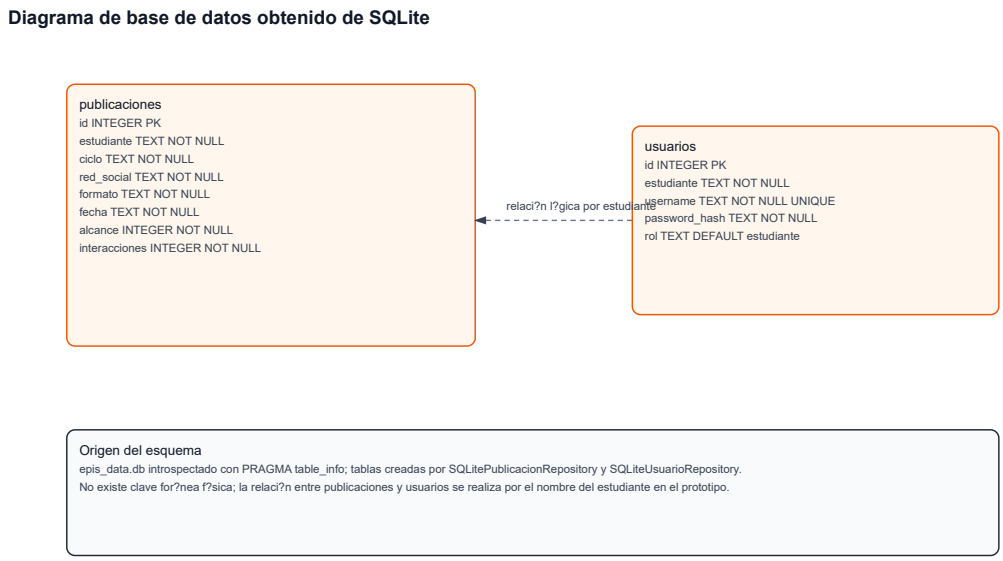


Tabla	Proposito
`publicaciones`	Almacena publicaciones simuladas, red social, ciclo, alcance e interacciones
`usuarios`	Almacena credenciales demostrativas y datos de estudiantes

5.7. Endpoints principales

Metodo	Endpoint	Descripcion	Seguridad
GET	`/api/health`	Verifica estado de la API	Publico
GET	`/api/estudiantes`	Lista estudiantes disponibles	Publico
POST	`/api/auth/login`	Autentica usuario y retorna token	Publico

Metodo	Endpoint	Descripcion	Seguridad
GET	`/api/dashboard/perfil`	Consulta metricas personales	Token
GET	`/api/dashboard/global`	Consulta metricas globales	Token

5.8. Evidencia de validacion

Las validaciones ejecutadas sobre la version final fueron:

Validacion	Comando	Resultado
Pruebas backend	`python -m pytest -q`	50 pruebas pasadas
Build frontend	`npm run build`	Compilacion correcta
Pruebas frontend	`npm test -- --watch=false`	2 pruebas pasadas
Terraform	`terraform validate`	Configuracion valida
Infracost	`infracost breakdown`	Total mensual base USD 9.192

5.9. RoadMap resumido

Fase	Descripcion	Estado
Fase 1	Prototipo local por capas y datos simulados	Completado
Fase 2	Separacion FastAPI + Angular	Completado
Fase 3	Login basico, token y rutas protegidas	Completado
Fase 4	Graficos reales, filtros y responsividad	Completado
Fase 5	Terraform, costos y documentacion final	Completado
Fase 6	Despliegue real cloud, PostgreSQL e integracion de datos reales	Pendiente

6. Cronograma

El cronograma se organiza por fases del proyecto academico:

Semana	Actividad	Entregable
1	Identificacion del problema, alcance y factibilidad	FD01
2	Vision del producto, usuarios, caracteristicas y Wiki	FD02
3	Historias de usuario, criterios y escenarios	FD03

Semana	Actividad	Entregable
4	Modelo, repositorios, servicios y pruebas backend	Codigo Python y pruebas
5	API FastAPI, endpoints y autenticacion	Backend web
6	Frontend Angular, vistas, login y dashboards	Aplicacion Angular
7	Diagramas desde codigo, base de datos e infraestructura	FD04
8	Terraform, Infracost, validaciones y consolidacion final	FD05

7. Presupuesto

7.1. Costos de desarrollo

Concepto	Costo estimado
Recursos generales del proyecto	S/ 180.00
Costos operativos directos	S/ 0.00
Ambiente base local	S/ 0.00
Personal del equipo del proyecto	S/ 4,100.00
Total estimado de desarrollo	**S/ 4,280.00**

7.2. Costos cloud estimados

Periodo	Costo USD	Costo aproximado en soles
1 mes	USD 9.192	S/ 34.47
6 meses	USD 55.15	S/ 206.81
12 meses	USD 110.30	S/ 413.63

Estos costos corresponden a una infraestructura minima propuesta para prototipo y no consideran aumentos por transferencia, trafico alto, dominios propios, monitoreo avanzado ni servicios administrados adicionales.

8. Conclusiones

1. EPIS Analytics cumple el objetivo de ofrecer un sistema web para visualizar indicadores simulados de actividad social de estudiantes EPIS mediante dashboard personal y dashboard global.

2. La evolucion desde Streamlit hacia FastAPI + Angular permitio una arquitectura mas separada, mantenible y cercana a un sistema web moderno.

3. El sistema considera los ciclos academicos I al X y redes sociales relevantes para el contexto inicial: LinkedIn, Instagram y YouTube.

4. La autenticacion con token, el guard de rutas y el interceptor Angular permiten proteger las vistas privadas del dashboard.

5. Las pruebas ejecutadas confirman estabilidad funcional en backend y frontend: 50 pruebas backend pasadas, 2 pruebas frontend pasadas y build Angular correcto.

6. La infraestructura propuesta en Terraform fue validada y el costo base estimado por Infracost es bajo para un prototipo academico: aproximadamente USD 9.192 mensuales.

7. Los documentos FD01, FD02, FD03 y FD04 mantienen trazabilidad con el producto final, permitiendo justificar factibilidad, vision, requerimientos y arquitectura.

9. Recomendaciones

1. Migrar SQLite a PostgreSQL si el sistema pasa a un entorno multiusuario o productivo.

2. Implementar roles diferenciados para estudiante, docente y administrador.

3. Incorporar recuperacion de contrasena y gestion segura de secretos si se habilita uso real.

4. Evaluar terminos de uso, permisos y consentimiento antes de integrar APIs reales de redes sociales.

5. Automatizar despliegue con CI/CD antes de ejecutar `terraform apply` en una cuenta cloud.

6. Agregar pruebas end-to-end para validar visualmente login, dashboard personal, dashboard global y filtros.

7. Considerar integracion futura con Power BI solo si se requiere analitica externa, gobierno de datos o reportes ejecutivos adicionales.

10. Bibliografia

- Universidad Privada de Tacna. Portal institucional de la UPT.
- Sommerville, I. *Ingenieria de Software*. Pearson.
- Pressman, R. *Ingenieria del Software: Un enfoque practico*. McGraw-Hill.
- Kimball, R. y Ross, M. *The Data Warehouse Toolkit*. Wiley.

11. Webgrafia

- Documentacion Angular: https://angular.dev/
- Documentacion FastAPI: https://fastapi.tiangolo.com/
- Documentacion SQLite: https://www.sqlite.org/docs.html
- Documentacion ECharts: https://echarts.apache.org/
- Documentacion Terraform: https://developer.hashicorp.com/terraform/docs
- Documentacion Infracost: https://www.infracost.io/docs/
- Documentacion AWS EC2: https://aws.amazon.com/ec2/
- Documentacion AWS S3: https://aws.amazon.com/s3/
- Documentacion AWS CloudFront: https://aws.amazon.com/cloudfront/

12. Anexos

Anexo 01. Informe de Factibilidad

Archivo relacionado: `FD01-Informe-Factibilidad.md`

Contenido principal:

- Analisis de riesgos.
- Factibilidad tecnica, economica, operativa, social, legal y ambiental.
- Infraestructura propuesta con Terraform.
- Analisis de costos con Infracost.
- Estimacion de desarrollo y costo cloud.

Anexo 02. Documento de Vision

Archivo relacionado: `FD02-Informe-Vision.md`

Contenido principal:

- Posicionamiento del producto.
- Usuarios e interesados.
- Vista general del producto.
- Contenido de GitHub Wiki.
- RoadMap del proyecto.

Anexo 03. Documento SRS

Archivo relacionado: `FD03-EPIS-Informe-Especificacion-Requerimientos.md`

Contenido principal:

- Contexto institucional UPT y EPIS.
- Requerimientos funcionales y no funcionales.
- Historias de usuario GitHub Issues HU-01 a HU-09.
- Criterios de aceptacion.
- Escenarios de prueba en formato Dado/Cuando/Entonces.
- Diagramas de secuencia relacionados.

Anexo 04. Documento SAD

Archivo relacionado: `FD04-EPIS-Informe-Arquitectura-de-Software.md`

Contenido principal:

- Arquitectura cliente-servidor.
- Diagramas obtenidos desde codigo.
- Diagrama de base de datos.
- Diagrama de infraestructura Terraform.
- Atributos de calidad.

Anexo 05. Manuales y otros documentos

Archivos relacionados:

- `README.md`
- `epis-web/README.md`
- `infra/terraform/README.md`
- GitHub Wiki del proyecto
- `arquitectura_epis.puml`

Estos anexos complementan la instalacion local, ejecucion del sistema, uso de endpoints, ejecucion de pruebas y mantenimiento del prototipo.